

```

1  """Week 11: audio feature engineering + classical ML baseline."""
2  from __future__ import annotations
3  from pathlib import Path
4  import numpy as np, pandas as pd, librosa
5  from scipy import signal
6  from sklearn.ensemble import RandomForestClassifier
7  from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
8  TARGET_SR=22050; DURATION=4.0; TARGET_LEN=int(TARGET_SR*DURATION); N_FFT=2048; HOP=512;
9  N_MELS=64; N_MFCC=20
10
11 def load_audio_fixed(path,target_sr=TARGET_SR,duration=DURATION):
12     y,sr=librosa.load(path,sr=None,mono=True)
13     if sr!=target_sr: y=librosa.resample(y,orig_sr=sr,target_sr=target_sr)
14     n=int(round(target_sr*duration)); y=np.pad(y,(0,max(0,n-y.size)))[0:n]
15     return y.astype(np.float32,copy=False)
16
17 def mel_db(y,sr=TARGET_SR):
18     M=librosa.feature.melspectrogram(y=y,sr=sr,n_fft=N_FFT,hop_length=HOP,n_mels=N_MELS,
19     power=2.0)
20     return librosa.power_to_db(M,ref=np.max)
21
22 def mfcc_matrix(y,sr=TARGET_SR): return librosa.feature.mfcc(S=mel_db(y,sr),n_mfcc=N_MFCC)
23
24 def mfcc_summary(y,sr=TARGET_SR):
25     C=mfcc_matrix(y,sr); return np.concatenate([C.mean(1),C.std(1)]).astype(np.float32)
26
27 def urbansound_path(root,row): return root/'audio'/f"fold{int(row['fold'])}"/str(row['slice_file_name'])
28
29 def featurize_rows(rows,root):
30     X=[]; y=[]
31     for _,row in rows.iterrows(): X.append(mfcc_summary(load_audio_fixed(urbansound_path(
32     root,row)))); y.append(row['class'])
33     return np.vstack(X),np.asarray(y)
34
35 def run_urbansound_baseline(root):
36     root=Path(root); meta=pd.read_csv(root/'metadata'/'UrbanSound8K.csv'); train=meta[
37     meta.fold.isin(range(1,9))]; valid=meta[meta.fold.eq(9)]; test=meta[meta.fold.eq(10)]
38     Xtr,ytr=featurize_rows(train,root); Xv,yv=featurize_rows(valid,root); Xte,yte=
39     featurize_rows(test,root)
40     clf=RandomForestClassifier(n_estimators=300,random_state=42,n_jobs=-1,class_weight=
41     'balanced_subsample').fit(Xtr,ytr)
42     print('Validation accuracy:',accuracy_score(yv,clf.predict(Xv))); yp=clf.predict(Xte
43     ); print('Test accuracy:',accuracy_score(yte,yp)); print(classification_report(yte,yp
44     )); return clf
45
46 if __name__=='__main__': print('Set a local UrbanSound8K path and call
47 run_urbansound_baseline(path).')

```